

GESTIÓN DE LA CONFIGURACIÓN

Para qué: Define dónde vive cada artefacto y cómo evoluciona. Cualquier integrante debe poder encontrar lo que necesita sin preguntar.

Para quién: Equipo, durante el año · Docente, para ubicar artefactos a revisar.

1. Estructura de repositorio y convención de nombres para documentos y código fuente

Para garantizar el orden del proyecto, se utiliza un enfoque monorrepo (o repositorios separados bajo una misma organización de GitHub) con la siguiente estructura y reglas de nomenclatura basadas en *kebab-case* para carpetas/archivos técnicos, y *PascalCase* para documentación formal.

Plaintext

```
aurora-project/
├── docs/                # Documentación de cátedra y diseño
│   ├── 01_EstudioInicial_Aurora.pdf
│   └── 02_UserStoryMap_Aurora.docx
├── src/                 # Código fuente del proyecto
│   ├── aurora/
│   │   ├── frontend/
│   │   └── backend/
└── README.md            # Guía de inducción y acceso rápido al project
```

- **Convención de nombres para código:** Carpetas y archivos en minúsculas separados por guiones (ej: `aurora`).
- **Convención de nombres para documentación:** Prefijo numérico de dos dígitos para indicar orden de lectura, seguido del nombre del entregable en CamelCase o PascalCase sin espacios (ej: `01_EstudioInicial_Aurora.pdf`).

2. Ciclo de vida de los artefactos: quién los crea, quién los actualiza y cuándo

Artefacto	Creador Inicial	Responsable de Actualizar	¿Cuándo se actualiza?
Estudio Inicial (Objetivos, Procesos, Impulsos y Propuesta)	Todo el Equipo	Rotativo por Sprint (Según asignación en Jira)	Al cierre de cada Sprint, únicamente si la cátedra solicita correcciones o si se redefine el alcance del problema a validar.
Plan de Proyecto (Metodología, WBS, Cronograma, Riesgos y Costos)	Todo el Equipo	Scrum Master del Sprint (Rotativo)	De manera mensual, para ajustar el cronograma ante desvíos reales respecto al calendario académico o mitigar nuevos riesgos técnicos detectados.
User Story Mapping y Backlog Inicial (Actividades, Tareas por actividad, Corte de MVP y Glosario)	Todo el Equipo	Product Owner del Sprint (Rotativo)	Al finalizar cada Sprint Planning o durante las sesiones de refinamiento de Backlog (<i>Grooming</i>), para detallar criterios de aceptación de las historias prioritarias.
Working Agreement (Normas, Compromisos, Roles Scrum y Canales)	Todo el Equipo	Todo el Equipo (Consenso general)	Obligatoriamente al inicio de cada Sprint durante la reunión de <i>Sprint Planning</i> , evaluando si las normas de convivencia y el uso de herramientas necesitan ajustes.

Gestión de la Configuración (Estructura de repositorio, Herramientas y Política de ramas)	Todo el Equipo	Integrador Técnico / DevOps del equipo	Al cierre de cada hito de entrega o si el equipo decide incorporar una nueva herramienta al ecosistema de desarrollo (ej: cambios en automatizaciones de n8n o esquemas de Supabase).
--	----------------	--	---

3. Herramientas utilizadas: repositorio, gestión de backlog, comunicación, CI/CD

El ecosistema de herramientas seleccionado para soportar el desarrollo ágil de Aurora se compone de:

- **Repositorio de Código y Documentación: GitHub.** Centraliza todo el historial de versiones del software y los entregables de la materia.
- **Gestión de Backlog: Jira Software.** Utilizado para crear los sprints, priorizar historias de usuario del MVP, asignar tareas y medir la velocidad del equipo mediante tableros Scrum.
- **Comunicación: Discord** (para weeklys) y **WhatsApp** (para coordinación interna rápida).

4. Política de ramas y proceso de revisión de pull requests

Se adopta una estrategia basada en **GitHub Flow** simplificada para asegurar la estabilidad del proyecto final:

- **Rama Principal (main):** Contiene el código completamente funcional, estable y listo para ser evaluado por la cátedra. Está protegida; nadie puede mergear directo.
- **Ramas de Características (feature/):** Cada tarea o historia de usuario se desarrolla en una rama propia que nace de **main**. Su nomenclatura sigue el formato: **feature/AUR-[Número_Tarea_Jira]-[Descripción_Breve]**. (Ejemplo: **feature/AUR-05-registro-paciente**).

Proceso de Revisión de Pull Requests (PR):

1. El desarrollador finaliza la tarea en su rama **feature/** y abre un **Pull Request** hacia **main**.
2. **Regla de Aprobación:** El PR requiere obligatoriamente la revisión y el aprobado de al menos **un miembro del equipo** ajeno a la creación de ese código (revisión por pares).
3. Una vez aprobado y verificado que no genera conflictos, se realiza el **Merge** hacia **main** y la rama de origen se elimina del repositorio remoto para mantener la limpieza del proyecto.